

University of Sheffield

Harmful Brain Activity Classification



Team Hotel

Qiyao Yang, Xin Quan, Zifan Wei, Aadit Thanekar, Shaguan Toleugali,
Lakshya Tripathi

Supervisor: Georgios Moulantzikos

COM6911 Team project

in the

Department of Computer Science

May 13, 2024

Declaration

All sentences or passages quoted in this document from other people's work have been specifically acknowledged by clear cross-referencing to author, work and pages. Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure.

Name: Team Hotel

Abstract

This project addresses the challenge of automatically detecting and classifying harmful brain activity in critically ill patients using electroencephalographic signals (EEG). Traditionally, this process relies on manual analysis by neurologists, which is time-consuming, expensive, and prone to fatigue and human error. This project explores the potential of deep learning models, specifically EfficientNetB2 and WaveNet, to automate this process and improve healthcare efficiency.

Contents

1	Introduction	1
1.1	The Human Brain: A Complex Network of Electrical Activity	1
1.1.1	The Kaggle Harmful Brain Activity Classification Challenge	1
1.2	Aims and Objectives	1
1.3	Overview of the Report	1
2	Background and literature review	3
2.1	Electroencephalography (EEG) and Spectrograms: Unveiling the Brain's Electrical Language	3
2.2	Machine Learning Approaches for Harmful Brain Activity Classification . . .	3
2.3	Limitations and Addressing the Gap	4
3	Research question	5
4	Methodology	6
4.1	Preprocessing Steps	6
4.2	Wavenet (using Raw EEG features)	7
4.2.1	Data Collection	7
4.2.2	Feature Engineering and Preparation	7
4.2.3	Model Development	8
4.2.4	Training and Performance Evaluation	8
4.3	EfficientNet	8
4.3.1	Model Architecture Introduction	8
4.3.2	Training Process	9
4.3.3	Performance Evaluation and Optimization	9
4.3.4	Cross Validation	10
5	Results	11
5.1	EfficientNet	11
5.2	Wavenet	11
6	Discussion and conclusions	13

List of Figures

List of Tables

5.1	Single model training results table	11
5.2	Cross-validation training results table	11
5.3	WaveNet Model training results table	12

Chapter 1

Introduction

1.1 The Human Brain: A Complex Network of Electrical Activity

The human brain is constantly actively generating electrical signals and impulses of different frequencies. Understanding these electrical patterns allows us to detect and classify different harmful brain activities, such as Seizures, Generalized Periodic Discharges, Lateralized Periodic Discharges, Lateralized Rhythmic Delta Activity, Generalized Rhythmic Delta Activity, and others. Early and accurate identification of these behaviors is a critical step in improving patient outcomes. Electroencephalography (EEG) is a fundamental diagnostic tool that measures the electrical activity of the human brain and is now widely used in medicine.

1.1.1 The Kaggle Harmful Brain Activity Classification Challenge

To address these limitations and accelerate advancements in automated brain activity classification, Kaggle, a platform for data science competitions, launched the Harmful Brain Activity Classification challenge. This competition provides a valuable platform for researchers to develop and test machine learning models capable of automatically identifying harmful brain activity patterns from EEG data.

1.2 Aims and Objectives

This research project aims to improve and understand pros and cons of two classifier models, EfficientNet and WaveNet, to identify harmful brain activities from EEG signals, thereby increasing the diagnostic reliability and accuracy in neurocritical settings.

1.3 Overview of the Report

The report is organized in the following manner: The background and literature review section is engaged in reviewing recent literature and research on harmful brain activity classification methods, including EfficientNetB2, and WaveNet, and provides brief information about EEG and spectrograms as representations of brain activity. The research questions addressed in the project are clearly stated in the Research Questions section. The detailed explanations

of two harmful brain activity classification models are covered in the Methodology part. The performance of these models is discussed and compared in the last part of the report.

Chapter 2

Background and literature review

2.1 Electroencephalography (EEG) and Spectrograms: Unveiling the Brain's Electrical Language

Electroencephalography (EEG) is a cornerstone technique for recording electrical activity generated by the brain. By placing electrodes on the scalp, EEG captures the summed electrical activity of large populations of neurons. This provides valuable insights into brain function but can be challenging to interpret due to its complex and time-series nature. To overcome these challenges, spectrograms are often used as a visual representation of EEG data. Spectrograms depict the distribution of electrical activity across different frequencies over time, offering a more intuitive way to identify patterns and changes in brain activity.

According to seminal research by Hawthorn et al. (2001), lorem ipsum dolor sit amet. However, this result was already known in the 1990s (Wieman & Hollberg 1991, Arnold et al. 1998).

2.2 Machine Learning Approaches for Harmful Brain Activity Classification

Recent years have witnessed a surge in the development of machine learning models for automated harmful brain activity classification. Deep learning architectures, with their ability to learn complex patterns from large datasets, have emerged as particularly promising tools.

EfficientNet: This state-of-the-art image recognition model has been successfully adapted for analyzing spectrograms. Its ability to capture subtle changes in frequency patterns makes it suitable for identifying potential markers of harmful brain activity.

CatBoost: This powerful gradient boosting algorithm excels at classification tasks and has been demonstrated effective in analyzing EEG data. Its flexibility and interpretability make it a valuable option for harmful brain activity classification.

WaveNet: This deep learning model specifically designed for sequential data excels at capturing temporal dependencies within EEG recordings. By effectively modeling the evolving nature of brain activity, WaveNet holds significant promise for accurate harmful brain activity classification.

2.3 Limitations and Addressing the Gap

While existing methods offer promising results, several limitations remain. Some models heavily rely on spectrograms, potentially losing valuable information present in the raw EEG signal. Additionally, some approaches lack interpretability, making it difficult to understand how they arrive at their classifications. This project aims to address these limitations by exploring a multifaceted approach. We will investigate the effectiveness of both EfficientNetB2 for spectrogram analysis and WaveNet for raw EEG data processing.

Chapter 3

Research question

How does the EfficientNet and WaveNet models compare against each other on the basis of correctly identifying harmful brain activity? The EfficientNet model only uses information from the provided 11138 spectrogram files whereas the Wavenet model trains on the EEG dataset. This report study the performances of the two models by analysing the accuracy and understand their strengths and weakness to determine which model is suitable and robust for detecting any kind of harmful activity in the brain. Furthermore, we will explore the potential of combining these models to leverage the strengths of each approach and potentially achieve superior classification performance.

Chapter 4

Methodology

4.1 Preprocessing Steps

Preprocessing of the spectrogram data is an essential phase for setting up the dataset in a way that maximizes the effectiveness of the deep learning model. The process is structured into three primary stages: initial data preparation, detailed transformation within the ‘SpectrogramDataset’ class, and a final normalization step.

Initial Data Preparation Instead of using a singular expert consensus, the preprocessing starts by normalizing the voting data where votes for each type of brain activity are aggregated and normalized by their total count. This method reflects the probabilistic nature of the output, where the model predicts probabilities for each activity category:

$$NormalizedVote_{ij} = \frac{Vote_{ij}}{\sum_{j=1}^n Vote_{ij}}$$

where $Vote_{ij}$ indicates the votes for the j -th brain activity type in the i -th spectrogram, and n represents the total number of activity types.

Spectrogram Dataset Transformation Within the ‘SpectrogramDataset’ class, significant transformations are applied to make the spectrogram data suitable for the neural network:

- **Handling Missing Values:** Missing values are filled with a constant value of -1 to ensure numerical stability throughout the dataset.
- **Data Reshaping and Conversion:** Spectrograms are initially oriented by frequency and need to be transposed to align correctly with the time dimension. This adjustment is critical as it affects how the neural network processes the time-series information:

$$Signal = Transpose(Spectrogram)$$

After transposition, the data is converted into a tensor format that is compatible with the neural network input requirements.

- **Uniform Input Sizing:** Due to varying sampling times, the spectrograms are resized to a uniform dimension of 512x512 pixels to standardize the input size and facilitate consistent processing across all data points.

Data Normalization for Model Input To prepare the data for a model trained on ImageNet, which originally handles three-channel images, the single-channel spectrogram tensor undergoes specific preprocessing:

$$Signal_{preprocessed} = \frac{\log(\text{clip}(Signal, e^{-6}, e^{10})) - \mu}{\sigma + \epsilon}$$

where μ and σ are the mean and standard deviation of the log-transformed signals, ensuring that the data is standardized with zero mean and unit variance, akin to the preprocessing of ImageNet training data.

Multithreading for Efficient Data Loading To enhance the efficiency of loading large spectrogram files, multiple threads are employed to handle I/O operations, significantly reducing the latency associated with reading data from disk.

Spectrogram Visualization

4.2 Wavenet (using Raw EEG features)

4.2.1 Data Collection

A publicly available dataset containing labeled spectrograms of EEG recordings was utilized. The dataset included patient IDs and recordings from patients with various harmful brain activity patterns and normal controls. Each recording is stored in a file called train.csv as a parquet file.

The EEG parquet recording, each being longer than 50 seconds, has a specific middle timestamp in seconds where the event occurring is predicted. This is to standardize the input size among different recordings. In the initial model from Kaggle, WaveNet Starter, all 20 features (EEG channels) are utilized to analyze brain activity. In the improved model, only eight specific EEG channels are selected. They are 'Fp1', 'F7', 'F3', 'O1', 'Fp2', 'F8', 'F4' and 'O2'.

4.2.2 Feature Engineering and Preparation

A Data Generator is employed in the preparation of data. It is implemented by using TensorFlow, 'tf.keras.utils.Sequence' class. This class provides that the model uses data in an optimized way. In the Data Generator class, by subtracting signals from one EEG channel from another, new features are created. This feature engineering method is called differential filtering. The new features show changes in electrical activity between brain regions. The next crucial step in the preparation of EEG data is standardizing. The newly created features are normalized by clipping outliers.

$$\text{np.clip}(\text{sample}, -1024, 1024)$$

The goal of this is to reduce the impact of outliers in the data, making model training stable and more efficient. In addition, replacing the NaN values with 0 and scaling by the standard deviation makes the training process stable and speeds up convergence.

4.2.3 Model Development

The model utilizes dilated convolutional layers to process temporal sequences, allowing the model to cover larger area of input data by skipping values at regular intervals. The model first uses 12 different convolutions with 8 channels each ($12 \times 8 = 96$ ways), spreading throughout the waveform using dilations. Each waveblock utilizes more channels and different convolutions using the previous waveblock information and continues to look more locally. The changing dilation rates is proportional to fluctuating computational costs. After processing the layers, a global average pooling is implemented to average out the features across each channel to focus on important information. The data is further processed through dense layers resulting to a softmax output layer which assigns probabilities and classifying the EEG data as 'Seizure', 'LPD', 'GPD' and others. The brain EEG channels refer to particular spatial areas (left-temporal, right parasagittal) are processed independently and then combined for accurate classification.

4.2.4 Training and Performance Evaluation

In the model, a Group K-Fold cross-validation strategy is utilized. This approach splits the training data set into 8 parts. Additionally, with this approach, the same patient data will be stored in one fold, which improves the model's ability to generalize data from unseen patients.

The model is constructed using the Kullback-Leibler divergence loss (KLD) function and the Adam optimizer. The Kullback-Leibler divergence loss evaluates the difference between the predicted and actual probability distributions of six classes. In addition, in terms of the optimizer, the Adam optimizer is implemented, since for each input parameter turning learning rate ensures that the model converges faster.

Also, in the training stage model's weights are stored in a .h5 file.

```
model.save_weights(f'WaveNet_Model/WaveNet_fold{i}.h5')
```

By saving weights, this allows it to be reused without retraining the model from the beginning. After saving weights, the model makes predictions and stores these predictions in the list `all_loof`. In this list, all predictions are stored for each fold.

After that step, the target values for the training and validation set are extracted to compute baseline prediction. In the subsequent for loop, the mean of each target class, in our case, there are 6 classes, is computed and then normalized to ensure that the sum of the prediction of each sample is equal to 1.

4.3 EfficientNet

4.3.1 Model Architecture Introduction

EfficientNet is an efficient convolutional neural network architecture known for its lightweight and high-performance characteristics. Its main architectural features include depth variabil-

ity, width variability, and resolution variability. In our task, we used EfficientNet as the base model, adjusting the input channel number and the classification layer to adapt to the specific task requirements.

Key layers of EfficientNet include convolutional layers, batch normalization layers, and activation function layers. The convolutional layers are responsible for extracting image features, batch normalization layers accelerate the training process and improve model stability, and activation function layers introduce non-linear characteristics, enhancing the model's representational capacity.

For our task, due to the specific number of categories in the neuroscience dataset, we needed to adjust the model's final classification layer to match the dataset's category count. Specifically, we set the `'num_classes'` parameter to 6 to match the number of categories in the dataset. Additionally, based on the dataset's characteristics, we also adjusted the input channel number to align with the dataset's features, thus better accommodating task requirements.

During model training, we supervised the model using the KL divergence loss function to minimise the difference between model predictions and true labels. Through such adjustments and configurations, we effectively utilised EfficientNet's efficiency and performance, achieving effective classification of the neuroscience dataset.

4.3.2 Training Process

The EfficientNet B0 model is created using the `timm` library, adjusting the classification layer to match six categories and ensuring the model can handle single-channel input in model creation. The `create_model` function decides whether to load pre-trained weights based on parameters and deploys the model to the specified device. The loss function uses KL Divergence, implemented through the `kl_divergence` function, which applies `log_softmax` transformation to the model predictions and calculates KL divergence with the true labels, normalizing using the batchmean method.

The `train` function is responsible for the training process. It begins by setting the model to training mode and then iterates through each epoch. For every batch within an epoch, the function performs forward propagation to obtain predictions from the model, calculates the loss using the KL Divergence function to measure the difference between predictions and true labels, and executes backpropagation to compute gradients. The optimizer is then used to update the model's parameters based on these gradients. After each optimizer step, the gradients are zeroed out to ensure clean gradients for the next batch. The average loss is calculated and printed at the end of each epoch to monitor training progress and ensure that the model is learning effectively.

4.3.3 Performance Evaluation and Optimization

The performance evaluation and optimization process of EfficientNet involved training a single model, followed by cross-validation to further refine the model. In the experiment, the single model was trained for three epochs, and signs of overfitting appeared in the third epoch, leading to the discontinuation of training. In the first epoch, the model achieved a private score of 0.587682 and a public score of 0.529739. This was the best score for the model on the private dataset. In the second epoch, the public score improved to 0.526433, which

was the best result for the model on the public dataset, while the private score deteriorated to 0.629905, indicating mixed performance between the two training epochs. In the third epoch, the private score increased to 0.682442, and the public score significantly increased to 0.610190, showing clear signs of overfitting, thus training was halted.

These three epochs yielded more significant results in the cross-validation experiment. In the first fold, the private score was 0.551153 and the public score was 0.495350. The second fold performed best, achieving a private score of 0.515403 and a public score of 0.478202. In the third fold, the private score was 0.557860 and the public score was 0.491273. The cross-validation results showed that the second fold was the best-performing model, demonstrating the value of cross-validation in identifying the optimal model and reducing overfitting.

4.3.4 Cross Validation

In order to fully evaluate the performance of the EfficientNet model on neuroscience datasets and to verify its generalization ability and stability, we used a cross-validation approach. In each fold, we divided the dataset into five mutually exclusive subsets, and then trained and validated each subset using different model configurations. By comparing the performance of different model configurations on different folds, we were able to select the best performing model configuration and use it as the final model. Not using cross-validation may affect the model effectiveness, which in turn leads to high KL scatter scores.

By training and validating on multiple data subsets, we can observe the performance of the model under different data distributions and thus verify the stability of the model. In addition, cross-validation is a means of data augmentation, which helps the model to better learn the distribution and characteristics of the data. Thus, cross-validation provides us with a comprehensive and reliable method to evaluate model performance, which provides scientific support and experimental basis for our research.

Chapter 5

Results

5.1 EfficientNet

Table 5.1 shows the training results when we do single model training.

Table 5.1: *Single model training results table*

Epoch	Private score	Public score	Result
1	0.587682	0.529739	private score best
2	0.629905	0.526433	public score best
3	0.682442	0.610190	overfitting

From the table, we can observe that a model trained until the third epoch, when a rise in scores is observed, indicating potential overfitting, can be stopped from further training. Incorporating cross-validation into our analysis allows for a comprehensive comparison of individual models across different epochs. This enables us to determine that models trained through an integrated approach, combining data from both the initial and subsequent epochs, yield superior results.

Table 5.2 shows the results of training after we added cross-validation.

Table 5.2: *Cross-validation training results table*

Epoch	Private score	Public score	Result
1	0.551153	0.495350	best model
2	0.515403	0.4782802	
3	0.557860	0.491273	

By comparing the scores obtained at the end, we can find that epoch 2 gives the best results, meaning that it is the best model.

5.2 Wavenet

The table below indicates how the model performed on different configurations of the available features and epochs. When the model utilizes all the features, the result shows signs of

overfitting with a very high score of 0.9127763, thereby reducing the number features suggests a slightly better generalization with a lesser score. Finally we train the model with 10 epochs and keep the same number of features resulting to the best score of 0.7147736. This shows that the model yielded the most generalized performance across unseen data.

Table 5.3: *WaveNet Model training results table*

Number of features	Epoch	Score	Result
20	5	0.9127763	overfitting
8	5	0.8107772	
8	10	0.7147736	best model

Chapter 6

Discussion and conclusions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Bibliography

Arnold, A. S., Wilson, J. S. & Boshier, M. G. (1998), ‘A simple extended-cavity diode laser’, *Review of Scientific Instruments* **69**(3), 1236–1239.

URL: <http://link.aip.org/link/?RSI/69/1236/1>

Hawthorn, C. J., Weber, K. P. & Scholten, R. E. (2001), ‘Littrow configuration tunable external cavity diode laser with fixed direction output beam’, *Review of Scientific Instruments* **72**(12), 4477–4479.

URL: <http://link.aip.org/link/?RSI/72/4477/1>

Wieman, C. E. & Hollberg, L. (1991), ‘Using diode lasers for atomic physics’, *Review of Scientific Instruments* **62**(1), 1–20.

URL: <http://link.aip.org/link/?RSI/62/1/1>